

Artificial Intelligence Engineering Project



Project: Retrieval Augmented Generation (RAG) LLM-based Application

Name: Richard Owusu

Course: Artificial Intelligence Engineering

Institution: Quantic School of Business and Technology

Date: July, 2026

Brief Description (Preliminaries)

This project delivers a **policy-focused RAG assistant** that answers employee questions using a curated company-policy corpus rather than generic model knowledge. It combines a Flask web interface, scope guardrails, retrieval over indexed policy chunks, and LLM-based answer generation with citations to improve transparency and trust.

During deployment, the work highlighted a key engineering tradeoff: local development can succeed while cloud free-tier environments fail due to memory/runtime constraints from heavy ML dependencies. The final approach emphasizes production practicality—lighter runtime settings, explicit app entrypoints, robust error handling, and deployment-aware configuration—so the assistant remains reliable, responsive, and suitable for demonstration and reporting.

Retrieval-Augmented Generation (RAG) Application Implementation/Features

The application was built as an end-to-end **Policy Q&A assistant** using a Retrieval-Augmented Generation (RAG) pipeline.

- A Flask backend was implemented with routes for `/` (UI), `/health` (service check), and `/chat` (question answering).
- A web interface was created to collect user questions and display generated answers plus citations.
- Policy documents were ingested, cleaned, chunked, and indexed into a vector store to support semantic retrieval.
- Guardrails were added to keep responses within the policy domain and reject out-of-scope questions.
- The `/chat` flow was designed to: validate input → run scope check → retrieve relevant chunks → generate answer via LLM → return citations/snippets.
- Response controls were added (e.g., top-k retrieval and answer trimming) to improve clarity and consistency.
- Error-handling and fallback responses were implemented so users receive stable outputs even when backend issues occur.
- Deployment configuration was iteratively fixed (entrypoint, build/runtime settings, environment variables) to move from local-only success toward cloud execution under resource constraints.

Overall, the work produced a functional, policy-grounded assistant with both user-facing UI and production-oriented backend safeguards.

Results Evaluation

The application was evaluated using functional, qualitative, and deployment-focused checks.

- **Functional testing:** Core endpoints (`/`, `/health`, `/chat`) were tested to confirm service availability, input validation, and response formatting.
- **Retrieval quality:** Queries related to company policies returned context-grounded answers with citations, showing that the RAG pipeline could retrieve and use relevant policy chunks.
- **Guardrail behavior:** Out-of-scope questions were correctly filtered, reducing hallucinated or irrelevant responses.
- **Response quality:** Answers were concise, policy-focused, and generally aligned with retrieved content; output trimming improved readability for end users.
- **Robustness:** Error-handling paths were validated so the UI remains stable during backend failures or timeout scenarios.

- **Deployment evaluation:** Cloud deployment revealed practical constraints (bundle size, memory limits, worker timeouts). These findings were used to improve runtime settings, dependency strategy, and deployment configuration.

Overall outcome: The system met its main objective of delivering grounded policy Q&A with citations, while the evaluation also identified infrastructure limitations and optimization opportunities for production-scale reliability.

1. Recorded demo presentation.

The recorded demo presentation of this project can be found here

https://drive.google.com/file/d/13zufYjjAfpLLqmmqAtNH_ugHR_8ziRC/view?usp=drive_link

2. GitHub repository

The RAG application was developed and tested locally; during the project life cycle a GitHub repository was created to track changes and host the final version. All updates were pushed to that repository - you can view the project GitHub repository

https://github.com/CobeRich/quantic_ai_project.

3. Summary of AI tooling

ChatGPT / AI assistant for:

- project scaffolding and task planning
- generating sample policies in html, md, pdf, and txt files
- improving documentation and submission-readiness

GitHub Copilot

- drafting initial implementations for ingestion, retrieval, and evaluation
- debugging environment/API errors
- for inline code suggestions and refactors.

These tools helped in:

- Faster troubleshooting for provider/API configuration errors
- Helped structure project to align with rubric requirements

- Accelerated documentation drafting for final submission

For manual Engineering:

- Environment and API key/quota debugging
- Provider switching and compatibility checks
- Data refresh pipeline (re-ingest/re-index flow)
- Metric interpretation and final quality validation
- End-to-end testing and endpoint behavior verification

Quality Control Processes:

- Ran local tests and endpoint checks
- Rebuilt index after corpus changes
- Executed evaluation and validated output artifacts (`results.csv`)
- Performed manual spot checks of answer grounding and citation relevance

4. Deployment/Production

The application was run in my local.